

Part 1: AI Lab Test - 2 (CSP)

1. CSP Representation

- **Variables:** $V = \{A, B, C\}$ representing the assigned (Time Slot, Lab) for each section.

PNG

- **Domains:**

- $D(A) = \{(1, L_1), (2, L_1)\}$

PNG

- $D(B) = \{(2, L_1), (2, L_2), (3, L_2), (4, L_2)\}$

PNG

- $D(C) = \{(1, L_2), (3, L_2), (4, L_2)\}$

PNG

- **Unary Constraints:**

- $T_A \in \{1, 2, 3\}, T_B \in \{2, 3, 4\}, T_C \in \{1, 3, 4\}$

PNG

- $L_A = L_1$

PNG

- $L_C \neq L_1 \implies L_C = L_2$

PNG

- L_1 is unavailable at $T = 3$.

PNG

- $T_B = 4 \implies L_B = L_2$

PNG

- $T_B = 4 \implies L_B = L_2$

PNG

- **Binary Constraints:**

- $T_A \neq T_B$

PNG

- $T_B \neq T_C$

PNG

- $|T_A - T_C| \neq 1$

PNG

- $(T_X = T_Y) \implies (L_X \neq L_Y)$ for any $X, Y \in \{A, B, C\}$

PNG

2-7. Search Sequence, MRV, and Forward Checking Trace

- **Initial State:** $D(A)$ size = 2, $D(B)$ size = 4, $D(C)$ size = 3. PNG + 1

- **Step 1:** Apply Minimum Remaining Values (MRV). Variable A is selected (2 values). PNG

- **Step 2:** Assign $A = (1, L_1)$. PNG

- **Step 3 (Forward Checking):**

- Check B : $T_B \neq T_A \implies T_B \neq 1$. No values removed. PNG + 1

- Check C : $|T_C - T_A| \neq 1 \implies T_C \neq 2$. No values removed. PNG + 1

- Domains: $D(B)$ size = 4, $D(C)$ size = 3. PNG

- **Step 4:** Apply MRV to $\{B, C\}$. Variable C is selected (3 values). PNG

- **Step 5:** Assign $C = (1, L_2)$. PNG

2-7. Search Sequence, MRV, and Forward Checking Trace

- **Initial State:** $D(A)$ size = 2, $D(B)$ size = 4, $D(C)$ size = 3. 
- **Step 1:** Apply Minimum Remaining Values (MRV). Variable A is selected (2 values). 
- **Step 2:** Assign $A = (1, L_1)$. 
- **Step 3 (Forward Checking):**
 - Check B : $T_B \neq T_A \implies T_B \neq 1$. No values removed. 
 - Check C : $|T_C - T_A| \neq 1 \implies T_C \neq 2$. No values removed. 
 - Domains: $D(B)$ size = 4, $D(C)$ size = 3. 
- **Step 4:** Apply MRV to $\{B, C\}$. Variable C is selected (3 values). 
- **Step 5:** Assign $C = (1, L_2)$. 
- **Step 6 (Forward Checking):**
 - Check B : $T_B \neq T_C \implies T_B \neq 1$. No values removed. 
 - Domains: $D(B)$ size = 4. 
- **Step 7:** Select variable B . 
- **Step 8:** Assign $B = (2, L_1)$. Forward checking validates all constraints. 
- **Condition Tracking:** No domains became empty, no constraints were violated, and no backtracking was required during this specific assignment path. 

8. Final Schedule

8. Final Schedule

Section	Time Slot	Lab
A	1	L_1
B	2	L_1
C	1	L_2
...		

CSP C++ Implementation

Csp implementation :-

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

using namespace std;

struct VarDomain {
    char name;
    vector<pair<int, int>> values;
};

bool checkConstraints(char v1, pair<int, int> val1, char v2, pair<int, int> val2) {
    if (val1.first == val2.first && val1.second == val2.second) return false;
    if ((v1 == 'A' && v2 == 'B') || (v1 == 'B' && v2 == 'A')) {
        if (val1.first == val2.first) return false;
    }
    if ((v1 == 'B' && v2 == 'C') || (v1 == 'C' && v2 == 'B')) {
        if (val1.first == val2.first) return false;
    }
}
```

```

        if ((v1 == 'A' && v2 == 'C') || (v1 == 'C' && v2 == 'A')) {
            if (abs(val1.first - val2.first) == 1) return false;
        }
        return true;
    }

void forwardCheck(vector<VarDomain>& unassigned, char currVar, pair<int,
int> currVal) {
    for (auto& var : unassigned) {
        vector<pair<int, int>> newVals;
        for (auto val : var.values) {
            if (checkConstraints(currVar, currVal, var.name, val)) {
                newVals.push_back(val);
            }
        }
        var.values = newVals;
    }
}

void solve(vector<VarDomain> unassigned, vector<pair<char, pair<int,
int>>> assigned) {
    if (unassigned.empty()) {
        cout << "Section | Time Slot | Lab\n";
        for (char c : {'A', 'B', 'C'}) {
            for (auto a : assigned) {
                if (a.first == c) {
                    cout << a.first << " | " << a.second.first << " | L"
<< a.second.second << "\n";
                }
            }
        }
        exit(0);
    }
    sort(unassigned.begin(), unassigned.end(), [](const VarDomain& a,
const VarDomain& b) {
        if (a.values.size() != b.values.size()) return a.values.size() <
b.values.size();
        return a.name > b.name;
    });
    VarDomain current = unassigned.front();

```

```

        unassigned.erase(unassigned.begin());

        for (auto val : current.values) {
            vector<pair<char, pair<int, int>>> newAssigned = assigned;
            newAssigned.push_back({current.name, val});
            vector<VarDomain> nextUnassigned = unassigned;
            forwardCheck(nextUnassigned, current.name, val);
            bool emptyDomain = false;
            for (auto& u : nextUnassigned) {
                if (u.values.empty()) emptyDomain = true;
            }
            if (!emptyDomain) {
                solve(nextUnassigned, newAssigned);
            }
        }
    }
}

int main() {
    vector<VarDomain> vars = {
        {'A', {{1, 1}, {2, 1}}},
        {'B', {{2, 1}, {2, 2}, {3, 2}, {4, 2}}},
        {'C', {{1, 2}, {3, 2}, {4, 2}}}
    };
    solve(vars, {});
    return 0;
}

```

Part 2: TSP using Genetic Algorithm

TSP C++ Implementation

```

#include <iostream>
#include <vector>
#include <cmath>
#include <numeric>
#include <random>
#include <algorithm>
#include <iomanip>

```

```

using namespace std;

struct City {
    int id;
    double x, y;
};

vector<City> cities = {
    {1, 60, 200}, {2, 23, 45}, {3, 15, 150}, {4, 85, 90}, {5, 71, 123},
    {6, 98, 45}, {7, 50, 220}, {8, 20, 30}, {9, 40, 180}, {10, 75, 155},
    {11, 82, 60}, {12, 91, 120}, {13, 12, 210}, {14, 37, 100}, {15, 28,
77},
    {16, 66, 89}, {17, 55, 130}, {18, 73, 140}, {19, 88, 33}, {20, 42,
170}
};

double distMatrix[20][20];

struct Individual {
    vector<int> tour;
    double distance;
    double fitness;
};

void computeDistanceMatrix() {
    for (int i = 0; i < 20; ++i) {
        for (int j = 0; j < 20; ++j) {
            double dx = cities[i].x - cities[j].x;
            double dy = cities[i].y - cities[j].y;
            distMatrix[i][j] = sqrt(dx * dx + dy * dy);
        }
    }
}

double calculateTotalDistance(const vector<int>& tour) {
    double total = 0;
    for (size_t i = 0; i < tour.size() - 1; ++i) {
        total += distMatrix[tour[i]][tour[i+1]];
    }
}

```

```

        total += distMatrix[tour.back()][tour.front()];
        return total;
    }

void evaluatePopulation(vector<Individual>& pop) {
    for (auto& ind : pop) {
        ind.distance = calculateTotalDistance(ind.tour);
        ind.fitness = 1.0 / ind.distance;
    }
    sort(pop.begin(), pop.end(), [](const Individual& a, const Individual&
b) {
        return a.fitness > b.fitness;
    });
}

Individual rouletteWheelSelection(const vector<Individual>& pop, mt19937&
gen) {
    double totalFitness = 0;
    for (const auto& ind : pop) totalFitness += ind.fitness;
    uniform_real_distribution<> dis(0, totalFitness);
    double r = dis(gen);
    double cumulative = 0;
    for (const auto& ind : pop) {
        cumulative += ind.fitness;
        if (cumulative >= r) return ind;
    }
    return pop.back();
}

Individual orderCrossover(const Individual& p1, const Individual& p2,
mt19937& gen) {
    int n = p1.tour.size();
    uniform_int_distribution<> dis(0, n - 1);
    int start = dis(gen);
    int end = dis(gen);
    if (start > end) swap(start, end);

    Individual offspring;
    offspring.tour.assign(n, -1);
    vector<bool> inOffspring(n, false);

```

```

        for (int i = start; i <= end; ++i) {
            offspring.tour[i] = p1.tour[i];
            inOffspring[p1.tour[i]] = true;
        }

        int p2Index = 0;
        for (int i = 0; i < n; ++i) {
            if (offspring.tour[i] == -1) {
                while (inOffspring[p2.tour[p2Index]]) {
                    p2Index++;
                }
                offspring.tour[i] = p2.tour[p2Index];
                inOffspring[p2.tour[p2Index]] = true;
            }
        }
        return offspring;
    }

void swapMutation(Individual& ind, mt19937& gen) {
    uniform_real_distribution<> prob(0.0, 1.0);
    if (prob(gen) < 0.1) {
        uniform_int_distribution<> dis(0, ind.tour.size() - 1);
        int idx1 = dis(gen);
        int idx2 = dis(gen);
        swap(ind.tour[idx1], ind.tour[idx2]);
    }
}

Individual runGA(int popSize, int numGenerations) {
    random_device rd;
    mt19937 gen(rd());

    vector<Individual> population(popSize);
    vector<int> baseTour(20);
    iota(baseTour.begin(), baseTour.end(), 0);

    for (int i = 0; i < popSize; ++i) {
        population[i].tour = baseTour;
    }
}

```

```

        shuffle(population[i].tour.begin(), population[i].tour.end(),
gen);
    }

    evaluatePopulation(population);

    for (int genIdx = 0; genIdx < numGenerations; ++genIdx) {
        vector<Individual> newPopulation;
        newPopulation.push_back(population[0]);

        while (newPopulation.size() < popSize) {
            Individual p1 = rouletteWheelSelection(population, gen);
            Individual p2 = rouletteWheelSelection(population, gen);
            Individual offspring = orderCrossover(p1, p2, gen);
            swapMutation(offspring, gen);
            newPopulation.push_back(offspring);
        }
        population = newPopulation;
        evaluatePopulation(population);
    }
    return population[0];
}

int main() {
    computeDistanceMatrix();

    vector<pair<int, int>> configs = {
        {10, 100}, {20, 100}, {30, 100},
        {10, 200}, {20, 200}, {30, 200}
    };

    for (auto config : configs) {
        int popSize = config.first;
        int gens = config.second;
        Individual best = runGA(popSize, gens);

        cout << "Population: " << popSize << ", Generations: " << gens
            << " | Distance: " << fixed << setprecision(2) <<
best.distance
            << " | Tour: ";

```

```
        for (int cityIdx : best.tour) {  
            cout << (cityIdx + 1) << " ";  
        }  
        cout << "\n";  
    }  
  
    return 0;  
}
```